CSE752: Deep Neural Networks

Week 2 Lecture Notes

Optimization and Regularization

Moin Moistakim
Department of Computer Science and Engineering
School of Data and Sciences, BRAC University

Fall 2026 | Slides: `CSE752_W02_Slides.tex` | https://cse.bracu.ac.bd/course/view/CSE752

How to use these notes

The slide deck is the in-class path. These notes hold the same updates with the algebra written out: the quadratic recurrence, the geometric series behind Adam, dropout's expectation, and batch-norm bookkeeping. Attempt Section 7 before Section 8.

Suggested timing for a 3-hour meeting. Quadratic gradient descent and divergence (25 min); SGD on two points (15 min); momentum and Nesterov (25 min); Adam and bias correction (30 min); schedules (15 min); overfitting, dropout, batch norm (40 min); problems (25 min); Home Assignment briefing (5 min).

Reading. Goodfellow, Bengio, Courville, *Deep Learning*, Ch. 7–8. Prince, *Understanding Deep Learning*, Ch. 6 and Ch. 11. Kingma and Ba (2015) for Adam; Srivastava et al. (2014) for dropout; Ioffe and Szegedy (2015) for batch normalization.

Convention. g denotes the gradient that the optimizer sees. For full-batch training that is ∇L. For a minibatch it is the average of the per-example gradients. PyTorch's parameter step is $w \leftarrow w - \eta\,(\text{processed } g)$, so a positive coordinate of g decreases the matching coordinate of w.

Contents

1 Gradient descent on a quadratic **1**
1.1 A two-dimensional valley . . . 2

2 Minibatch SGD **2**

3 Momentum and Nesterov **3**

4 Adam and AdamW **3**
4.1 Coupled decay versus AdamW . . . 4

5 Learning-rate schedules **4**

6 Overfitting, dropout, batch normalization **5**
6.1 The gap . . . 5
6.2 Dropout . . . 5
6.3 Batch normalization . . . 5

7 Problems **6**

8 Solutions **7**

---

1 Gradient descent on a quadratic

Definition 1.1 (Gradient descent)**.** For a differentiable L and a step size $\eta > 0$,

$$w_{t+1} = w_t - \eta\,\nabla L(w_t). \tag{1}$$

Throughout Sections 1–4 the scalar model is

$$L(w) = \frac{1}{2}(w-4)^2, \qquad L'(w) = w-4, \qquad L''(w) = 1, \tag{2}$$

unless a section says otherwise. The unique minimizer is $w^\star = 4$.

Proposition 1.1 (Error recurrence)**.** *Under* (1) *and* (2)*,*

$$w_{t+1} - 4 = (1-\eta)(w_t - 4).$$

Hence $w_t - 4 = (1-\eta)^t(w_0 - 4)$*. The error tends to* 0 *if and only if* $|1-\eta| < 1$*. For* $\eta > 0$ *this is* $0 < \eta < 2$*.*

Proof.

$$w_{t+1} = w_t - \eta(w_t - 4) = (1-\eta)w_t + 4\eta,$$

so

$$w_{t+1} - 4 = (1-\eta)w_t + 4\eta - 4 = (1-\eta)(w_t - 4).$$

The closed form is the iteration of that linear map. A geometric sequence r^t tends to 0 iff $|r| < 1$. □

The bound $\eta < 2$ is the special case of the general smooth-convex restriction $\eta < 2/L_{\text{smooth}}$ when the largest Hessian eigenvalue is $L_{\text{smooth}} = 1$. On a neural-network loss the Hessian changes along the path; the quadratic still tells you what "too large" looks like.

Example 1.1 (Three step sizes)**.** Start at $w_0 = 0$.

- $\eta = \frac{1}{2}$. Errors multiply by $\frac{1}{2}$: $w = (0,\, 2,\, 3,\, 3.5, \ldots)$, since $w_1 = 2$, $w_2 = 3$, $w_3 = 7/2$.
- $\eta = 1$. Then $1 - \eta = 0$, so $w_1 = 4$. One step reaches the minimizer. This is Newton's method for a quadratic with Hessian 1.
- $\eta = 3$. $w_1 = 12$, $w_2 = 12 - 3 \cdot 8 = -12$, $w_3 = -12 - 3 \cdot (-16) = 36$. The error grows and flips sign.

1.1 A two-dimensional valley

Example 1.2 (Conditioning)**.**

$$L(w_1, w_2) = \frac{1}{2}w_1^2 + 50\, w_2^2, \qquad \nabla L = (w_1,\, 100 w_2).$$

The Hessian eigenvalues are $\lambda_1 = 1$ and $\lambda_2 = 100$. The condition number is 100. Stability along the sharp axis requires $\eta < 2/100$. At $\eta = 0.01$ and $w = (1, 1)$,

$$w_1 \leftarrow 1 - 0.01 \cdot 1 = 0.99, \qquad w_2 \leftarrow 1 - 0.01 \cdot 100 = 0.$$

The factor on the flat axis is $1 - \eta\lambda_1 = 0.99$. After 100 steps of this size, if the gradient stayed linear, the flat error would be $0.99^{100} \approx e^{-1.005} \approx 0.366$. Any η large enough to move w_1 quickly makes w_2 diverge. Coordinate-wise scaling of the step is the motivation for the second-moment methods in Section 4.

2 Minibatch SGD

Let $L(w) = \frac{1}{N}\sum_{i=1}^{N} L_i(w)$. A minibatch B produces

$$g_B(w) = \frac{1}{|B|}\sum_{i \in B} \nabla L_i(w), \qquad w \leftarrow w - \eta\, g_B(w). \tag{3}$$

If B is drawn uniformly, $\mathbb{E}[g_B] = \nabla L$. The variance of g_B is of order $1/|B|$ for independent summands. Full-batch gradient descent is (3) with $B = \{1, \ldots, N\}$.

Example 2.1 (Two points)**.**

$$L(w) = \frac{1}{4}\sum_{i=1}^{2}(w - y_i)^2, \quad y = (1, 3).$$

Then $L'(w) = w - 2$, so $w^\star = 2$. Per-example derivatives are $w - y_i$. At $w = 0$ with $\eta = 1$:

- Full batch: $g = \frac{1}{2}\big((-1) + (-3)\big) = -2$, and $w \leftarrow 2$.
- Stochastic, example 1 then example 2: first $g = -1$ gives $w = 1$; then $g = 1 - 3 = -2$ gives $w = 3$.

The stochastic iterate overshoots $w^\star$. Later steps can come back. The noise is the reason a moderate learning rate is used for many epochs rather than one giant Newton step, and it is also the reason two runs with different shuffles are not identical.

Home Assignment fixes `batch_size=128` and one seed so that optimizer comparisons are not confounded with a new shuffle pattern each run. The seed does not remove minibatch noise; it makes that noise the same noise.

3 Momentum and Nesterov

Definition 3.1 (PyTorch SGD with momentum)**.** With buffer $v_0 = 0$ and coefficient $\mu \in [0, 1)$,

$$v_{t+1} = \mu v_t + g_t, \qquad w_{t+1} = w_t - \eta\, v_{t+1}. \tag{4}$$

Proposition 3.1 (Equivalence with Goodfellow's buffer)**.** *Write $u_{t+1} = \mu u_t - \eta g_t$ and $w_{t+1} = w_t + u_{t+1}$, with $u_0 = 0$. Then $u_t = -\eta v_t$ for every t, and the two sequences of w agree.*

Proof. The claim holds at $t = 0$. If $u_t = -\eta v_t$, then

$$u_{t+1} = \mu(-\eta v_t) - \eta g_t = -\eta(\mu v_t + g_t) = -\eta v_{t+1},$$

and $w_t + u_{t+1} = w_t - \eta v_{t+1}$. □

So a discrepancy between a textbook formula and a PyTorch run is often the placement of η, not a different algorithm. Compare trajectories of w, not the raw buffers.

Example 3.1 (Momentum hits the minimizer in two steps)**.** Use (2), $\eta = \mu = \frac{1}{2}$, $w_0 = 0$, $v_0 = 0$.

$$\begin{aligned} g_0 &= -4, & v_1 &= -4, & w_1 &= 2, \\ g_1 &= -2, & v_2 &= \tfrac{1}{2}(-4) - 2 = -4, & w_2 &= 4. \end{aligned}$$

Plain gradient descent with the same η has $w_2 = 3$ (Example 1.1). The second velocity still contains half of the first gradient, so the step does not shrink just because the current gradient shrank.

Definition 3.2 (Nesterov, PyTorch form)**.** After the gradient g has been computed at the current w,

$$v \leftarrow \mu v + g, \qquad w \leftarrow w - \eta\,(g + \mu v). \tag{5}$$

Classical momentum uses $w \leftarrow w - \eta v$ instead of the second line.

The usual derivation evaluates ∇L at the lookahead point $w - \eta\mu v$. PyTorch folds that idea into (5) so the training loop can still call `backward` at the current weights. For the assignment, `nesterov=True` is this second line and nothing else.

Example 3.2 (Nesterov on the same quadratic)**.** Continue with $\eta = \mu = \frac{1}{2}$, $w_0 = 0$, $v_0 = 0$.

$$\begin{aligned} v_1 &= -4, & g_0 + \mu v_1 &= -6, & w_1 &= 3, & \\ g_1 &= -1, & v_2 &= -3, & g_1 + \mu v_2 &= -\tfrac{5}{2}, & w_2 &= \tfrac{17}{4}. \end{aligned}$$

Step 1 lands closer to 4 than classical momentum (3 versus 2). Step 2 overshoots (4.25 versus the exact hit at 4). Nesterov is a more aggressive use of the same buffer, not a guarantee of fewer steps on every loss.

4 Adam and AdamW

Definition 4.1 (Adam)**.** Buffers $m_0 = v_0 = 0$, coefficients $\beta_1, \beta_2 \in [0, 1)$, and $\varepsilon > 0$. For $t \geq 1$,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t, \tag{6}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, (g_t \odot g_t), \tag{7}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \tag{8}$$

$$w_t = w_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}. \tag{9}$$

The square root and the quotient are coordinate-wise. Defaults used in the assignment: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\eta = 10^{-3}$.

Proposition 4.1 (Bias of the first moment)**.** *Unfold* (6)*. If $g_1 = \cdots = g_t = g$, then $m_t = (1 - \beta_1^t)\, g$, so $\hat{m}_t = g$.*

Proof. $m_0 = 0$ gives

$$m_t = (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g = g\,(1 - \beta_1) \sum_{j=0}^{t-1} \beta_1^j = g\,(1 - \beta_1) \cdot \frac{1 - \beta_1^t}{1 - \beta_1} = g\,(1 - \beta_1^t).$$

Divide by $1 - \beta_1^t$. □

The same expansion with $g_i \odot g_i$ and β_2 gives $\hat{v}_t = g \odot g$ when the squared gradient is constant. Early in training β_1^t is not negligible, so using m_t in place of $\hat{m}_t$ shrinks the step. For $\beta_1 = 0.9$, the raw buffer at $t = 1$ is only 10% of g.

If the gradients are random with constant mean, the same identity holds with g replaced by $\mathbb{E}[g]$, because expectation passes through (6). The correction is exact for that mean; it is not a claim that every noisy g_t equals $\hat{m}_t$.

Example 4.1 (One fraction-friendly Adam step)**.** Let $g = 2$, $\beta_1 = \beta_2 = \frac{1}{2}$, $\eta = \frac{1}{5}$, $\varepsilon = 0$, $t = 1$.

$$m_1 = 1, \quad v_1 = 2, \quad \hat{m}_1 = 2, \quad \hat{v}_1 = 4, \quad \frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = 1, \quad \Delta w = \frac{1}{5}.$$

The uncorrected ratio is $m_1/\sqrt{v_1} = 1/\sqrt{2}$, and the step would be $1/(5\sqrt{2}) \approx 0.141$. At step 1 the correction multiplies m by 2 and v by 2, and the square root turns the second factor into $\sqrt{2}$, so the ratio grows by $\sqrt{2}$.

4.1 Coupled decay versus AdamW

PyTorch `Adam`'s `weight_decay` replaces g by $g + \lambda w$ *before* (6)–(9). On a scalar, at step 1, with $\varepsilon = 0$,

$$\tilde{g} = g + \lambda w, \qquad \frac{\hat{m}}{\sqrt{\hat{v}}} = \frac{\tilde{g}}{|\tilde{g}|} = \operatorname{sign}(\tilde{g}),$$

so the update moves by η in that sign and the value of λ cancels. AdamW (Loshchilov and Hutter, 2019) leaves the moments on the raw gradient and subtracts $\eta\lambda w$ afterwards:

$$w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon} - \eta\lambda w. \tag{10}$$

Example 4.2 (The cancellation, numerically)**.** Take $w = 2$, $g = 1$, $\lambda = 0.1$, $\eta = 0.1$, step 1, $\varepsilon = 0$.

- Coupled: $\tilde{g} = 1.2$, ratio $= 1$, step $= 0.1$, new weight 1.9.
- AdamW: adaptive step 0.1, decay term $0.1 \cdot 0.1 \cdot 2 = 0.02$, new weight 1.88.

The two updates agree in the special case $\sqrt{\hat{v}} = 1$ and the decay is added to a moment that is not then renormalized. They disagree as soon as the second moment is not 1, which is the usual situation. Home Assignment's optional stretch, AdamW at 10^{-4}, is this decoupled term.

Weight decay and dropout are not substitutes. Decay pulls parameters toward the origin on every step. Dropout (next section) multiplies hidden activations by random masks and does not change the set of parameters.

5 Learning-rate schedules

Both schedules below replace η and leave the optimizer's buffers alone.

Step decay. `StepLR(step_size=5, gamma=0.1)` with initial $\eta = 0.1$ uses η, then 0.01, then 0.001, at blocks of five scheduler steps. In the assignment, `sched.step()` runs once per epoch, after `opt.step()`. The first five epochs therefore still train at 0.1.

Cosine.

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2}\left(1 + \cos\frac{\pi t}{T}\right). \tag{11}$$

With $\eta_{\max} = 0.1$, $\eta_{\min} = 0$, $T = 8$: $\eta_0 = 0.1$, $\eta_4 = 0.05$, $\eta_8 = 0$. The formula does not look at the loss. It is a predetermined curve.

A schedule is not a regularizer in the sense of Section 6. It changes how long the optimizer is allowed to take large steps. Early stopping, which *is* a regularizer, chooses the number of steps from the validation loss rather than from a formula in t.

6 Overfitting, dropout, batch normalization

6.1 The gap

On a fixed architecture, a smaller training set makes the minimizer of the training loss a worse predictor of the test loss. Home Assignment's 1,000-image, 25-epoch Adam run is designed to make that visible: plot training accuracy and test accuracy on one figure. A gap that widens while training accuracy climbs is overfitting. Reporting only the training curve hides it.

Regularizers used this week:

- **Weight decay,** Section 4 and the quadratic penalty $\frac{\lambda}{2}\|w\|^2$, whose gradient contribution is λw. One SGD step multiplies w by $1 - \eta\lambda$.
- **Dropout,** random hidden masks during training.
- **Batch normalization,** which changes the optimization geometry more than it changes the set of representable functions (the γ, β pair can represent the un-normalized scale).
- **Early stopping,** freeze the weights at the best validation epoch.

6.2 Dropout

Definition 6.1 (Inverted dropout)**.** For a hidden vector h and a drop probability $p \in (0, 1)$, draw m_i i.i.d. Bernoulli with $¶(m_i = 1) = q = 1 - p$, and set

$$h'_i = \frac{m_i}{q}\, h_i \qquad \text{during training.} \tag{12}$$

At evaluation, $h' = h$.

Proposition 6.1. *Under Definition 6.1,* $\mathbb{E}[h'_i \mid h] = h_i$.

Proof. $\mathbb{E}[m_i] = q$, so $\mathbb{E}[m_i h_i / q \mid h] = h_i$. □

The factor $1/q$ is the reason evaluation must not also drop units and must not multiply by $1/q$. Training already matched the expectation to h.

Example 6.1 (One mask)**.** $p = \frac{1}{2}$, so the scale is 2. Let $h = (2, 0, 4, 6)$ and $m = (1, 0, 1, 0)$. Then $h' = (4, 0, 8, 0)$. A second forward pass draws a new mask. That is why test-time dropout makes accuracy a random variable. `model.eval()` disables the mask; `model.train()` restores it. The assignment's comparison of the train/test gap is meaningless if those modes are swapped.

6.3 Batch normalization

Definition 6.2 (Batch norm, one unit, training)**.** On a batch $x_1, \dots, x_m$,

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2, \tag{13}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta. \tag{14}$$

The variance in (13) divides by m (the biased estimator). γ and β are parameters.

Example 6.2 (Four activations)**.** $x = (1, 3, 5, 7)$, $\varepsilon = 0$, $\gamma = 1$, $\beta = 0$.

$$\mu_B = 4, \qquad \sigma_B^2 = \frac{9 + 1 + 1 + 9}{4} = 5, \qquad \hat{x} = \frac{(-3, -1, 1, 3)}{\sqrt{5}}.$$

The coordinates of $\hat{x}$ sum to 0, and their mean square is 1.

Running statistics. PyTorch updates buffers with momentum $\alpha = 0.1$ by default:

$$\mu_{\text{run}} \leftarrow (1 - \alpha)\mu_{\text{run}} + \alpha\, \mu_B, \tag{15}$$

$$\sigma^2_{\text{run}} \leftarrow (1 - \alpha)\sigma^2_{\text{run}} + \alpha\, \sigma^2_{\text{unbiased}}, \tag{16}$$

where the unbiased batch variance is $\frac{m}{m-1}\sigma_B^2$ (Bessel's correction). The forward pass during training still uses σ_B^2 from (13). At evaluation,

$$\hat{x} = \frac{x - \mu_{\text{run}}}{\sqrt{\sigma^2_{\text{run}} + \varepsilon}},$$

then the same affine map (14). A single test image has no batch whose mean should define the network, and a test loader's batch size is an evaluation choice, not part of the trained function. Cold running statistics (the first batches, while μ_{run} is still near its zero initialization) are a practical reason not to stop at epoch 1 and export the buffers.

Placement. The assignment uses `Linear(784, 128)`, `BatchNorm1d(128)`, `ReLU`. The layer that is normalized is the 128-dimensional pre-activation. Batch norm couples the examples inside a minibatch: μ_B and σ_B^2 depend on every row, so the gradient for one row depends on the other rows. That coupling is absent at test time, which is another reason the two modes must not share code paths accidentally.

SGD at $\eta = 0.3$ is a probe of this geometry. Without the normalization, a step that large often diverges on this MLP; with it, the pre-activations are reset to unit scale every batch and the same step can be stable. The ablation table should record which of those two things you observed, not a generic claim that batch norm "always helps accuracy."

7 Problems

P1. For $L(w) = \frac{1}{2}(w - 4)^2$ and $w_0 = 0$, compute the gradient-descent iterate w_1 at $\eta = 1$, and w_1, w_2 at $\eta = 3$. For which $\eta > 0$ does $w_t \to 4$?

P2. On Example 1.2, start at $(1, 1)$ with $\eta = 0.01$. Compute one gradient step. Which coordinate has moved farther, and which restriction on η forced the small step?

P3. Repeat Example 3.1's setup but with $\eta = \frac{1}{10}$ and $\mu = \frac{9}{10}$. Compute w_1 and w_2 for momentum, and w_2 for plain gradient descent. Which is closer to 4?

P4. Using Example 2.1, write the stochastic path of length 2 and the full-batch result after one step. Why can the stochastic path leave $w^\star$ even though each per-example gradient is computed correctly?

P5. Prove Proposition 4.1. Then, with $g = 2$, $\beta_1 = \beta_2 = \frac{1}{2}$, $\eta = \frac{1}{5}$, $\varepsilon = 0$, compute the Adam step at $t = 1$ and the uncorrected step.

P6. Take $w = 2$, $g = 1$, $\lambda = \eta = 0.1$, step 1 of scalar Adam with $\varepsilon = 0$. Compute the updated weight for coupled weight decay and for AdamW.

P7. State inverted dropout. Prove $\mathbb{E}[h'_i \mid h] = h_i$. Apply one mask $(1, 0, 1, 0)$ to $h = (2, 0, 4, 6)$ with $p = \frac{1}{2}$. What does `model.eval()` change, and what fails if dropout stays on at test time?

P8. For the batch $(2, 4, 4, 6)$, compute μ_B, biased σ_B^2, and $\hat{x}$ with $\varepsilon = 0$, $\gamma = 1$, $\beta = 0$. If μ_{run} starts at 0 and $\alpha = 0.1$, compute μ_{run} after this batch. Which statistics are used at test time?

8 Solutions

P1

$\eta = 1$ gives $w_1 = 4$. $\eta = 3$ gives $w_1 = 12$ and $w_2 = -12$. By Proposition 1.1, $w_t \to 4$ for $0 < \eta < 2$.

P2

$\nabla L(1, 1) = (1, 100)$. One step: $(0.99,\ 0)$. The sharp coordinate w_2 moved by 1; the flat coordinate moved by 0.01. The restriction $\eta < 2/100$ is what keeps w_2 from diverging, and it is also what makes w_1 crawl. See Example 1.2.

P3

Momentum: $g_0 = -4$, $v_1 = -4$, $w_1 = \frac{2}{5}$. Then $g_1 = -\frac{18}{5}$ and

$$v_2 = \tfrac{9}{10}(-4) - \tfrac{18}{5} = -\tfrac{36}{5}, \qquad w_2 = \tfrac{2}{5} + \tfrac{18}{25} = \tfrac{28}{25}.$$

Plain GD: $w_1 = \frac{2}{5}$, $w_2 = \frac{19}{25}$. Distances to 4 are 72/25 (momentum) and 81/25 (GD). Momentum is closer.

P4

Full batch, one step: $w = 2$, which is $w^\star$. Stochastic path: $0 \to 1 \to 3$. Each per-example derivative $w - y_i$ is the gradient of that example's loss, not of the average loss. Subtracting $w - y_1$ moves toward $y_1 = 1$, not toward the mean 2. The next example then moves toward 3. Unbiasedness says the *expected* update matches ∇L; a single sample path does not.

P5

The proof is Proposition 4.1. The numbers are Example 4.1: corrected step $\frac{1}{5}$, uncorrected step $1/(5\sqrt{2})$.

P6

Coupled decay: $\tilde{g} = 1.2$, and $\hat{m}/\sqrt{\hat{v}} = \operatorname{sign}(1.2) = 1$, so the weight moves by 0.1 and becomes 1.9. AdamW: adaptive move 0.1 plus $\eta\lambda w = 0.02$, so the weight becomes 1.88. This is Example 4.2.

P7

Definition 6.1 and Proposition 6.1. The masked output is $(4, 0, 8, 0)$. `model.eval()` replaces (12) by the identity. Leaving dropout on at test time draws a new subnetwork every forward pass, so the reported accuracy is random, and the $1/q$ scale that training relied on is not the scale of a deterministic network.

P8

$\mu_B = 4$, squared deviations $4, 0, 0, 4$, so $\sigma_B^2 = 2$, and

$$\hat{x} = \frac{(-2, 0, 0, 2)}{\sqrt{2}} = (-\sqrt{2},\ 0,\ 0,\ \sqrt{2}).$$

$\mu_{\text{run}} = 0.4$. Test time uses μ_{run} and σ^2_{run}, not μ_B of a training batch and not the mean of whatever batch the test loader happened to form.

CSE752 Week 2 lecture notes. Companion slides: `lectures/CSE752_W02_Slides.tex`. Home Assignment: `home_assignment/CSE752_W02_Home_Assignment.tex`.